

API Authentication Guide

REST API v2 — Authentication and Authorization

Complete guide to authenticating with the platform API, including OAuth 2.0 flows, API key management, JWT handling, and rate limiting.

VERSION
2.1

STATUS
ACTIVE

DATE
15 January 2026

AUTHOR
Engineering Team

Contents

- 1 Overview 2
- 2 Quick Start 2
 - 2.1 Getting API Keys 2
 - 2.2 Your First Request 2
- 3 Authentication Flows 2
 - 3.1 Client Credentials Flow 2
 - 3.2 Authorization Code Flow 3
- 4 JWT Token Structure 3
 - 4.1 Verifying Tokens 3
- 5 Rate Limiting 3
- 6 Error Handling 4
 - 6.1 Common Error Codes 4
- 7 Security Best Practices 4

1 Overview

The API uses **OAuth 2.0** for authentication and **JWT tokens** for session management. All API endpoints require authentication unless explicitly marked as public.

All API requests must be made over HTTPS. Calls made over plain HTTP will be rejected. API requests without authentication will return `401 Unauthorized`.

2 Quick Start

2.1 Getting API Keys

1. Log in to the Developer Portal
2. Navigate to **Settings > API Keys**
3. Click **Generate New Key**
4. Store your `client_id` and `client_secret` securely

2.2 Your First Request

```
# Exchange credentials for an access token
curl -X POST https://api.example.com/v2/auth/token \
-H "Content-Type: application/json" \
-d '{
  "client_id": "your_client_id",
  "client_secret": "your_client_secret",
  "grant_type": "client_credentials"
}'
```

The response contains your access token:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "read write"
}
```

3 Authentication Flows

3.1 Client Credentials Flow

Best for **server-to-server** communication where no user context is needed.

```
import requests

def get_access_token(client_id, client_secret):
    """Exchange client credentials for an access token."""
    response = requests.post(
        "https://api.example.com/v2/auth/token",
        json={
            "client_id": client_id,
            "client_secret": client_secret,
            "grant_type": "client_credentials",
        },
    )
    response.raise_for_status()
    return response.json()["access_token"]
```

3.2 Authorization Code Flow

Best for **user-facing applications** where you need to act on behalf of a user.

1. Redirect the user to the authorization endpoint
2. User grants permission
3. Exchange the authorization code for tokens
4. Use the access token for API calls

```
package main

import (
    "fmt"
    "net/http"
)

func handleCallback(w http.ResponseWriter, r *http.Request) {
    code := r.URL.Query().Get("code")
    if code == "" {
        http.Error(w, "Missing authorization code", 400)
        return
    }

    // Exchange code for token
    token, err := exchangeCode(code)
    if err != nil {
        http.Error(w, "Token exchange failed", 500)
        return
    }

    fmt.Fprintf(w, "Authenticated! Token: %s...", token[:20])
}
```

4 JWT Token Structure

Access tokens are **RS256-signed JWTs** with the following claims:

Claim	Type	Description
sub	string	Subject (user or client ID)
iss	string	Issuer (authorization server URL)
aud	string	Audience (API base URL)
exp	number	Expiration time (Unix timestamp)
iat	number	Issued at (Unix timestamp)
scope	string	Space-separated list of granted scopes

4.1 Verifying Tokens

Always verify tokens server-side. The public key is available at:

```
https://auth.example.com/.well-known/jwks.json
```

5 Rate Limiting

API calls are rate-limited per access token:

Plan	Requests/minute	Requests/day
Free	60	10,000
Pro	600	100,000
Enterprise	6,000	Unlimited

Rate limit headers are included in every response:

- `X-RateLimit-Limit` – maximum requests per window
- `X-RateLimit-Remaining` – requests remaining
- `X-RateLimit-Reset` – Unix timestamp when the window resets

6 Error Handling

All errors follow RFC 7807 (Problem Details for HTTP APIs):

```
{
  "type": "https://api.example.com/errors/invalid-token",
  "title": "Invalid Access Token",
  "status": 401,
  "detail": "The access token has expired. Please refresh your token.",
  "instance": "/v2/users/me"
}
```

6.1 Common Error Codes

Status	Error	Resolution
401	invalid-token	Refresh or re-authenticate
403	insufficient-scope	Request additional scopes
429	rate-limit-exceeded	Wait and retry with backoff

7 Security Best Practices

- **Never expose credentials in client-side code.** Use a backend proxy for browser applications.
- **Rotate API keys regularly.** Set up automated key rotation on a 90-day cycle.
- **Use the minimum required scopes.** Follow the principle of least privilege.
- **Implement token refresh.** Do not wait for 401 errors – refresh proactively.
- **Log authentication events.** Monitor for unusual patterns and failed attempts.

For questions or issues, contact the API support team or open a ticket in the Developer Portal.